

TRABAJO
INTERMODULAR:
Sistemas Informáticos

Sistemas Informáticos

1º DAM

Hecho por Raúl Herranz, Iñigo Rojo y Nacho de León

Índice

1. Introducción
2. Entorno de ejecución de aplicaciones Java
 - 2.1 La Máquina Virtual de Java (JVM)
 - 2.2 Diferencia entre JDK y JRE
 - 2.3 Instalación y comprobación en Windows y Linux
 - 2.4 Variables de entorno: JAVA_HOME y PATH
3. Creación de instaladores para aplicaciones Java
 - 3.1 ¿Qué es un instalador y por qué no usar un .zip?
 - 3.2 Herramientas para crear instaladores
 - 3.3 Qué lleva un instalador de verdad por dentro
4. Servicios, APIs y comunicación en red
 - 4.1 ¿Qué es una API? (El modelo camarero)
 - 4.2 Spring Boot: El "todo en uno" de Java
 - 4.3 Conceptos de red: IPs, puertos y peticiones
5. Contenedores con Docker
 - 5.1 El problema del "en mi ordenador funciona"
 - 5.2 Máquina Virtual vs. Contenedor
 - 5.3 Conceptos básicos de Docker
 - 5.4 Java y Docker juntos
6. Aplicaciones frontend con Angular y React
 - 6.1 ¿Qué papel juega el Frontend?
 - 6.2 Node.js y npm: El entorno del frontend
 - 6.3 El proceso de Build
 - 6.4 Desplegando en un servidor web
 - 6.5 Cómo habla el Frontend con el Backend Java

1. Introducción

Pasar de tener un proyecto en IntelliJ o Eclipse a que esa aplicación funcione de verdad en un servidor o en el ordenador de un cliente es tarea de Sistemas Informáticos.

El objetivo de este trabajo no es escribir código, sino entender todo lo que pasa por detrás. Vamos a ver qué necesita un ordenador para ejecutar Java, cómo se empaqueta de forma profesional, cómo se conecta a otros sitios mediante APIs, cómo Docker nos salva la vida en los despliegues y cómo encaja todo esto con las páginas web modernas hechas en Angular o React.

2. Entorno de ejecución de aplicaciones Java

2.1 La Máquina Virtual de Java JVM

Si en Windows haces doble clic en un archivo .exe compilado para Windows, funciona. Si lo intentas en Mac, no. Java no funciona así. Cuando compilamos un programa Java, no creamos código que el procesador entienda directamente, sino algo llamado bytecode.

Para leer ese bytecode entra en juego la JVM Java Virtual Machine. Básicamente es un traductor que se instala en tu sistema operativo. Le dice al Windows: ey, este código dice que haga esto, hazlo, y al Linux: ey, a ti te dice lo mismo, hazlo a tu manera. Gracias a la JVM, el programador escribe el código una vez y corre en cualquier sitio.

2.2 Diferencia entre JDK y JRE

Esto es de las preguntas clásicas. Cuando te bajas Java, hay dos opciones principales: JRE Java Runtime Environment Es lo que necesita tu vecino o tu madre para ejecutar el programa que tú has hecho. Solo tiene lo justo para correr la aplicación JVM y las librerías básicas.

JDK Java Development Kit es lo que necesitamos nosotros, los que programamos. Lleva todo lo del JRE, pero además incluye el compilador javac, el gestor de paquetes y otras herramientas de desarrollo. Para programar hace falta el JDK, pero para usar un programa solo hace falta el JRE.

2.3 Instalación y comprobación en Windows y Linux

Instalar Java es bastante sencillo. En Windows te bajas un instalador y le das a Siguiente sin pensar mucho aunque hay que vigilar la ruta. En Linux por ejemplo Ubuntu, abres la terminal y usas el gestor de paquetes: `sudo apt install openjdk-17-jdk`

Para comprobar que no te has vuelto loco y está bien instalado, abres la terminal en Windows el CMD o PowerShell y escribes:

```
java -version
```

Si te devuelve la versión que te acabas de instalar, perfecto. Si te dice java no se reconoce como un comando interno o externo, es que algo falló en la configuración del sistema.

2.4 Variables de entorno JAVA_HOME y PATH

Este es el punto donde más dolores de cabeza da a los que empiezan. Las variables de entorno son como notas que le deja el sistema operativo a sí mismo para recordar dónde están las cosas.

JAVA_HOME Le dice al sistema Oye, Java se ha instalado en esta carpeta exacta ej C:\Program Files\Java\jdk-17. Hay programas como Maven o servidores que buscan esta variable para saber dónde está Java.

PATH Es una lista de carpetas donde el sistema busca ejecutables automáticamente. Si escribes java en la terminal y funciona, es porque la carpeta bin de Java está en el PATH. Si no lo está, el sistema se queda mirando a la pared y no encuentra el comando.

3. Creación de instaladores para aplicaciones Java

3.1 ¿Qué es un instalador y por qué no usar un .zip?

Imagina que te descargas un juego nuevo y en lugar de un instalador te dan una carpeta comprimida en .zip con 500 archivos sueltos y una nota que dice busca el archivo .jar y ejecútalo, pero antes asegúrate de tener la versión 17 de Java. Pésimo, ¿verdad?

Un instalador es un programa que hace todo ese trabajo sucio por ti de forma automática. Sus ventajas son claras da una imagen profesional, puede instalar el JRE por debajo sin que el usuario se entere, crea accesos directos en el escritorio y, muy importante, genera un desinstalador para dejar el PC limpio si ya no quieres el programa.

3.2 Herramientas para crear instaladores

No tenemos que programar un instalador desde cero, hay herramientas muy buenas:

Viene incluida en el JDK moderno. Es la opción oficial de Java. Le pasas tu .jar y te genera un .exe o .msi en Windows o un .deb en Linux listo para distribuir.

Launch4j Muy usada en Windows. Lo que hace es envolver tu .jar dentro de un .exe de verdad. Así tu programa tiene su icono propio y el usuario no tiene que lidiar con la extensión .jar.

Inno Setup No es solo para Java, pero es de los más potentes en Windows para crear instaladores con pantallas de bienvenida, barras de progreso y selección de carpeta.

3.3 Qué lleva un instalador de verdad por dentro

Un buen instalador no solo copia y pega archivos. Suele incluir:

1. Los archivos de tu aplicación el .jar, imágenes, etc.
2. El JRE embebido para que el usuario no tenga que instalar nada.
3. Opciones de configuración inicial ej preguntar en qué puerto quieres que corra.
4. La creación de accesos directos en el escritorio o menú de inicio.
5. Escrituras en el registro de Windows para asociar archivos por ejemplo.
6. Un desinstalador que borre todo lo que se instaló.
7. Servicios, APIs y comunicación en red

4. Normas del trabajo

4.1 ¿Qué es una API? El modelo camarero

En el modelo cliente servidor, una API Interfaz de Programación de Aplicaciones es el puente que conecta dos programas que están en máquinas distintas.

La mejor forma de entenderlo es imaginar un restaurante tú eres el cliente tu móvil o

navegador, la cocina es el servidor donde está la base de datos y la lógica pesada de Java, y la API es el camarero. Tú le pides la carta al camarero petición, él va a la cocina, te trae la comida respuesta y tú no tienes ni idea de cómo se ha cocinado. Normalmente, las APIs modernas hablan en un formato de texto llamado JSON.

4.2 Spring Boot El todo en uno de Java

Antes, para hacer una API en Java tenías que instalar un servidor externo como Tomcat, configurarlo, exportar tu proyecto a un formato especial llamado .war y subirlo. Era un rollo.

Spring Boot cambió las reglas del juego. Es un framework que te permite hacer una API y, lo más guay, te la empaqueta en un simple archivo .jar que lleva el servidor web integrado dentro. Para el administrador de sistemas, desplegar esto en un servidor Linux es increíblemente fácil si subes el .jar, abres la terminal y escribes `java -jar mi-api.jar`. El servidor arranca solo.

4.3 Conceptos de red IPs, puertos y peticiones

Para que la API del camarero funcione, la red tiene que estar bien configurada:

Dirección IP es la dirección física o lógica del servidor en internet como el número de la calle de un restaurante.

Un servidor puede tener muchas APIs o servicios a la vez. El puerto es como la mesa del restaurante. Spring Boot por defecto abre el puerto 8080. Si intentas entrar por el 8081, no hay nadie.

Protocolo HTTP HTTPS es el idioma en el que hablan. HTTPS es igual pero con un candado de seguridad fundamental hoy en día.

Peticiones GET y POST son las dos formas principales de pedir cosas. GET es para pedir información y POST es para mandar información nueva. El servidor siempre contesta con un código 200 si todo ok, 404 si no lo encuentra, 500 si el servidor ha explotado.

5. Contenedores con Docker

5.1 El problema del en mi ordenador funciona

Seguro que has oído el meme En mi ordenador funciona perfectamente. Esto pasa porque el ordenador del programador tiene Windows, Java 17 y una librería instalada, pero el servidor de producción tiene Linux, Java 11 y no tiene la librería.

Docker es la solución a esto. Empaqueta tu aplicación junto con todo lo que necesita para funcionar en una caja cerrada y aislada. Si funciona en tu Docker en casa, funcionará en el servidor de Amazon o Google exactamente igual.

5.2 Máquina Virtual vs Contenedor

No hay que confundir Docker con una Máquina Virtual como VirtualBox

Máquina Virtual Es como comprar un PC nuevo dentro de tu PC. Tiene su propio Windows o Linux completo. Gasta muchísima RAM y tarda un minuto en encenderse.

Contenedor: Comparte el sistema operativo de tu PC, pero aísla la aplicación. Es súper ligero, gasta poquísima RAM y arranca en un segundo. Puedes tener 10 contenedores corriendo a la vez sin que tu PC sufra.

5.3 Conceptos básicos de Docker

Imagen Es la foto estática de tu aplicación con todo lo que necesita. Es el molde.

Contenedor Es la imagen puesta a funcionar. Es como la copia viva del molde.

Dockerfile Es un archivo de texto donde le dices a Docker cómo construir la imagen
ejemplo Usa Linux, instala Java 17, copia mi archivo jar, ejecútalo Si borras un contenedor, se borran sus datos. Un volumen es una carpeta de tu ordenador real conectada al contenedor para que la base de datos no se borre si reinicias Docker.

Puerto expuesto Conectar el puerto interno del contenedor con un puerto de tu ordenador real para poder acceder desde el navegador.

5.4 Java y Docker juntos

Hoy en día, un administrador de sistemas no instala Java en los servidores para desplegar una app de Spring Boot. Lo que hace es crear una imagen Docker que ya lleva un mini Linux y Java preinstalado, mete dentro el jar de Spring Boot y lo ejecuta. Así de simple.

6. Aplicaciones frontend con Angular y React

6.1 ¿Qué papel juega el Frontend?

Hasta ahora hablábamos del backend Java, APIs y servidores. Pero alguien tiene que hacer las pantallas de botones, los formularios de login y las listas bonitas. Ahí entra el Frontend. Es la parte de la aplicación que se ejecuta en el navegador del usuario. Su único trabajo es verse bien, capturar lo que hace el usuario y pedirle los datos a la API de Spring Boot.

6.2 Node.js y npm El entorno del frontend

Aquí hay algo curioso: Angular y React se programan en JavaScript, un lenguaje que nació para funcionar dentro del navegador web. ¿Cómo vamos a ejecutar herramientas de Angular en nuestro ordenador si no hay navegador? Para solucionar esto se creó Node.js, que es como un navegador sin interfaz gráfica que te permite ejecutar JavaScript en tu PC.

Y claro, si hay librerías, hay que gestionarlas. Aquí entra npm Node Package Manager, que es el equivalente a Maven en Java. Cuando escribes npm install, se baja de internet todas las librerías que necesita tu proyecto React o Angular.

También tenemos el servidor de desarrollo cuando estás programando, escribes npm start y se abre una pestaña en localhost 3000. Lo importante es que si cambias algo y guardas, el navegador se actualiza automáticamente.

6.3 El proceso de Build

Cuando terminas el proyecto, no puedes subir el código fuente de Angular o React al servidor de producción. Primero hay que hacer un Build escribiendo npm run build.

Este proceso coge todo el código, lo optimiza, elimina espacios innecesarios, lo comprime y lo deja listo para producción.

El resultado es una carpeta que contiene archivos HTML CSS y JavaScript listos para funcionar sin necesidad de Node.js.

6.4 Desplegando en un servidor web

Esa carpeta generada hay que subirla a un servidor web como Nginx o Apache.

El servidor se encarga de enviar esos archivos al navegador del usuario de forma rápida.

6.5 Cómo habla el Frontend con el Backend Java

El frontend y el backend se comunican mediante peticiones HTTP.

El proceso es

1. El usuario entra en la web
2. El frontend envía una petición al backend
3. El backend procesa la información
4. Devuelve una respuesta
5. El frontend muestra los datos

Un punto importante es CORS Cross Origin Resource Sharing. Los navegadores bloquean peticiones entre dominios distintos por seguridad. Por eso hay que configurar el backend para permitir las peticiones desde el frontend.